

An Empirical Study on the Requirements Engineering Practices for Agile Software Development

Mohamad Kassab
Engineering Division
The Pennsylvania State University
Malvern, PA, U.S.A
muk36@psu.edu

Abstract— Collecting, understanding, and managing requirements are critical aspects in all development methods including agile methods as well. Nevertheless, little contemporary data exists for document actual practices of software professionals for software requirements engineering activities in agile environments. To remedy this deficiency and provide useful data to other researchers we conducted a survey study that drew participants from wide range of professions, industries and geographic locations. We filtered the survey responses according to the Software Development Life Cycle; then we analyzed how agile processes (compared to traditional waterfall model) deal with a paradox of requirements engineering. In this paper, we present this exploratory survey and its quantitative results.

Keywords- *Requirements Engineering; agile development; common practices; software development industry; software professionals.*

I. INTRODUCTION

Increasing competition and complex business processes are pushing IT firms gain competitive advantage by squeezing delivery timelines [1]. Companies are thus increasingly seeking ways to become more responsive and flexible in their methodology and requirements. Agile development practices have become widely accepted as an effective class of approaches to project management in order to have rapid delivery of high-quality software [2]. Usually the work of the teams adopting an agile methodology is flexible and changeable. In comparison to traditional software processes, agile development is less document-centric and more code-oriented, adaptive rather than predictive, and people-oriented rather than process-oriented [3].

Nevertheless, requirements engineering (RE) discipline was challenged under the agile umbrella. This is basically true because “doing requirements” requires spending more time studying the business and application world and less time programming. Doing requirements takes time—time that could be spent writing code—and users often feel better if they think that people are “working” (that is, coding) on their problems [4]. In fact, ever since agile first emerged in the 1990s, there has been debate about what role, RE can play for agile practitioners and their customers [5]. Williams discusses in [6] some of the shortcomings in using agile methodologies with respect to RE.

Little contemporary data exists for document actual practices of software professionals for software RE activities in agile environments. To remedy this deficiency and provide useful data to other researchers we conducted a survey study on the current RE state of practice. The survey was created as a web-based survey using the web-based QuestionPro survey tool (www.QuestionPro.com); and survey data was collected through April and July 2013. The survey consisted of 32 questions; and a summary of the survey questions may be viewed through the link: goo.gl/HzMO7U

Participants of the survey were drawn from multiple sources: 1) A database of former students in the Masters of Software Engineering degree program at the Penn State Great Valley School of Graduate Professional Studies (PSGV). PSGV caters primarily to working professionals; 2) Subscribers to the IEEE Reliability Society newsletter, which has a circulation of >3000 professional members; 3) Members of the following Linked-In professional groups: Requirements Engineering Specialist Group (RESG), Computing Reviews Reviewers, CSIAC Software Intensive Systems Engineering (formerly DACS), DAMA (data managers association)/Philadelphia region, DAMA (data managers association)/International, IEEE Computer Society members, Software Engineering, IEEE –USA.

The survey drew 247 participants from 23 countries (with a completion rate of 48%). The survey responses captured a diverse mix of industries, job roles, project domains and RE practices. Respondents were asked to base all their project responses on one project only that they were either currently involved with or had taken part in during the past five years.

In this paper we filtered the responses according to the Software Development Life Cycle (SDLC) that was followed in the participants’ projects; then we focused on analyzing how agile processes (compared to the traditional waterfall processes) deals with a paradox of requirements engineering.

It is important to make a distinction between RE for agile methodologies and “agile RE”. Agile RE means, generally, any ad hoc RE approach purported to be more flexible than “traditional” RE. This definition not be confused with specific practices for RE in agile methodologies; which is the scope of this paper.

The remaining of this paper is organized as follows: Section II provides the background on the survey participants and the characteristics of the surveyed projects. Section III presents the results on RE practices in agile projects

compared to the traditional waterfall projects. Section IV discusses related work and finally, Section V concludes the paper.

II. SURVEY PARTICIPANTS AND PROJECTS' CHARACTERISTICS

In order to understand the types of characteristics of respondents, a number of questions regarding "Organizational Characteristics" were asked.

Overall, survey participants reflected a diverse range of positions describing themselves as programmers / developers, software / system engineers, or testers 61% of the time. Architects, project / product managers, analysts, and consultants comprised the remaining 39% of respondents. Respondents also reported a wide range of project experience. When asked how many projects they had worked on in the last five years answers ranged from 2 to more than 50. The mean experience level in this survey in terms of number of projects in the last five years is 22.57; and the median experience level is 17.5. The standard deviation of 16.25 is indicative the broad spectrum of experience levels reported. Nevertheless; and in order to ensure consistency; respondents were asked to base all their project responses on one project only that they were either currently involved with or had taken part in during the past five years. Several questions asked respondents then to classify and categorize the relevant project.

Overall, 46% of respondents indicated that they used an agile methodology (e.g. SCRUM, Extreme Programming, Feature Driven Development, Lean) as SDLC for their project making agile the most popular SDLC (Figure 1). The usage of agile is almost doubled since 2008 when a similar survey was conducted [7].

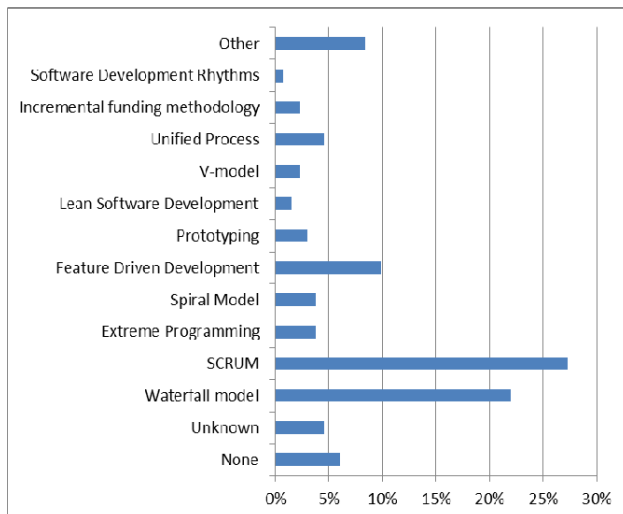


Figure 1. Software Development Lifecycle employed

Further analysis linked this question's response to both the respondent's type of industry and geographic location. We observed that the agile development is more used than the waterfall model in all industries except in the "finance /

banking/ insurance" industry where the waterfall methodology is still slightly more practiced (see Figure 2). In some particular industries like gaming and sales, agile practice was the dominant practice. We also observed that waterfall model was barely used in medical and telecommunications industries.

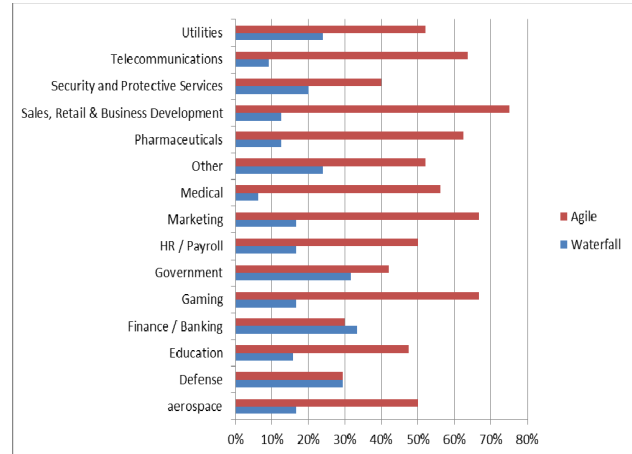


Figure 2. Software Development Lifecycle employed across industries

As for the linkage to the geographic location, we noticed that the practice of the waterfall model outside the United States is lower than its usage in the United States in general. We also noticed that agile practice is more prevalent than the waterfall practice in every region in the United States except of the "Northeast", where agile and waterfall are equally popular (see Figure 3). Given the banking and insurance hubs in New York and Delaware, This might also account for the apparent prevalence of waterfall development in the Northeast.

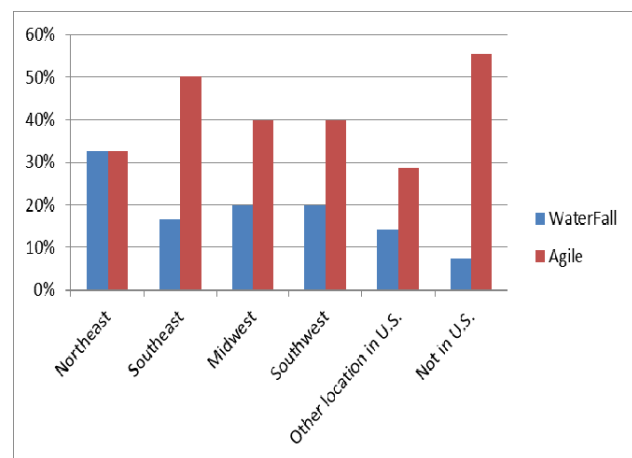


Figure 3. Software Development Lifecycle employed across geographic locations

With respect to project's schedules, the majority of the projects (59%) were a year or less in duration. Another 21% took between 1 to 2 years to complete and only 19% of the reported projects took more than 2 years to complete. For

those projects that followed an agile development method, 43% of these projects took less than 10 iteration to complete, 14% took between 11 to 20 iterations to complete, and only 9% of these projects took more than 20 iterations to complete. On average each iteration spanned 22 business days.

Technical debt is a metaphor referring to the eventual consequences of poor software architecture and software development within a codebase. Ward Cunningham coined the metaphor back in 1992 [8]. The debt can be thought of as work that needs to be done before a particular job can be considered complete. When asked how much technical debt on average was carried from one iteration or sprint to the next, 91% of the respondents reported carrying some level of technical debt but the majority of the participants (84%) reported that this level is below 25% on average. In other words, for this majority of participants, when a new iteration or sprint started; there was on average 25% of a work still to be completed from previous iterations or sprints.

Fifty six percent of projects comprised 50,000 LOC or less, which we would characterize as small to medium in scale. Remembering that agile projects focuses on small and medium size projects [9], it was a surprise to find that for the reported responses on large projects in this survey (> 50,000 LOC), agile practices outstripped the waterfall model (26% compared to 14%).

III. REQUIREMENTS ENGINEERING PRACTICES IN AGILE VS. WATERFALL

In varying form, it is common belief that agile methods use high-level and low-ceremony requirements practices [6]. Customers are constantly involved in requirements discovery and refinement in agile methods. All systems developers are involved in the RE activity and each can and should have regular interaction with customers. In traditional approaches, the customers has less involvement once the requirements specification has been written and approved, and typically, the involvement is often with the systems developers. Presumably, the agile approach to RE is much more invulnerable to changes throughout the process than in traditional software engineering.

In this section, we explore and present selected data on RE practices in agile environments from the survey, along

with comparison with the reported RE practices in the waterfall environments. The RE focus areas of this section are: elicitation, analysis and presentation, management, effort estimation, and tools.

A. Requirements Elicitations

Requirements elicitation tries to discover requirements and identify system boundaries by consulting stakeholders (e.g., clients, developers, users). There are several requirement techniques available for requirements gathering. We presented an extensive list of known RE elicitation techniques; participants could select all that applied for the reported projects. Respondents reported using, on average, 2.26 requirements gathering techniques. The data (see Figure 4) revealed that “Brainstorming” was the most common approach for both agile and waterfall projects. Brainstorming consists of informal sessions with customers and other stakeholders to generate overarching goals for the systems.

“User Stories” ranked second most popular technique for agile projects. A user story represents a feature customers want in the software [10]. User stories are written by a customer, maybe with the assistance of the developers.

In [1], the authors suggested that the project using agile should learn from various interviewing techniques that have been in use in the RE of waterfall. We indeed observe that for agile projects, Interviews was a common elicitation technique as for the waterfall projects as well (ranked third for agile and second for waterfall).

While in a past survey, “Scenarios” was the most popular method employed overall (at 50% in 2002) [11], we observe that the usage of “Scenarios” is declining (34% reported its usage in this survey; dropping down to the 6th position overall). Scenarios are informal descriptions of the system in use that provide a high-level description of system operation, classes of users, and exceptional situations. In agile projects “Scenarios” was less used than in the waterfall projects. Scenarios were ranked the 3rd most used technique in waterfall projects while it ranked 7th in the agile projects. The dominance of agile methodological use reported in this survey likely explains the decline of the “Scenarios” technique from the one presented in 2002 when waterfall dominated [11].

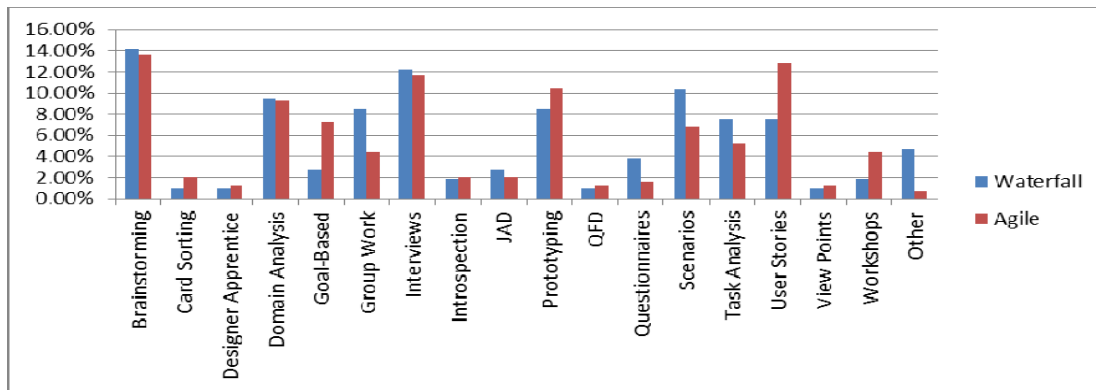


Figure 4. Requirements gathering techniques used across agile vs. waterfall

B. Requirements Analysis and Representation

Requirements Analysis checks requirements for necessity (the need for the requirement), consistency (requirements should not be contradictory), completeness (no service or constraint is missing), and feasibility (requirements are feasible in the context of the budget and schedule available for the system development). Conflicts in requirements are resolved through prioritization and negotiation with stakeholders [12].

As Object Oriented (OO) Analysis is a technique often applies along with the use of “Scenarios”; and with the decline of “Scenarios” we expected a lower proportion using OO analysis techniques compared to last survey (30% in 2002) [11]. The participants’ responses surprisingly revealed that 38% of this survey population reported using OO analysis. In fact, OO analysis was the most popular technique employed in both waterfall and agile projects (See Figure 5).

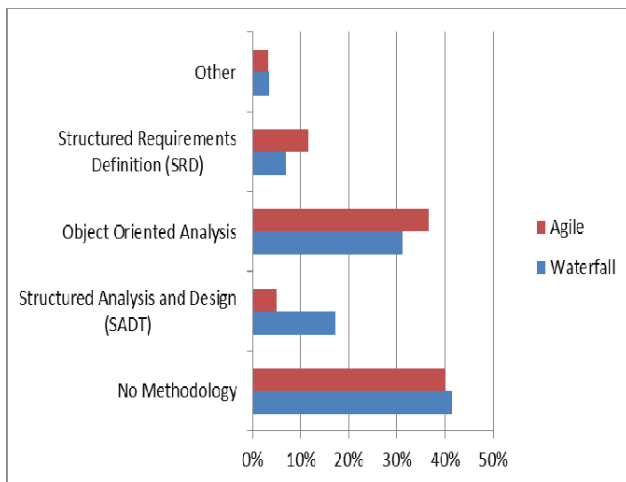


Figure 5. Requirements analysis techniques used across agile vs. waterfall

When asked “how satisfied were with the requirements analysis and modeling efforts”; 60% of agile projects reported a level of satisfaction – compared to 36% for the waterfall.

An interesting corollary to the requirements modeling and analysis is the requirements representation’s degree of formality. The majority of users (61.16%) still report that requirements are expressed in terms of natural language. Natural language was the dominant technique for both agile and waterfall projects (See Figure 6). We see that only 33% of users reported utilizing semi-formal notations such as UML. When we filtered this number across waterfall vs. agile projects; we observed that 37% of agile projects used UML notations (comparing to only 21% for waterfall projects). This is consistent with the number of respondents who reported using OO analysis techniques. It is also of interest that formal specifications techniques are still not commonly utilized.

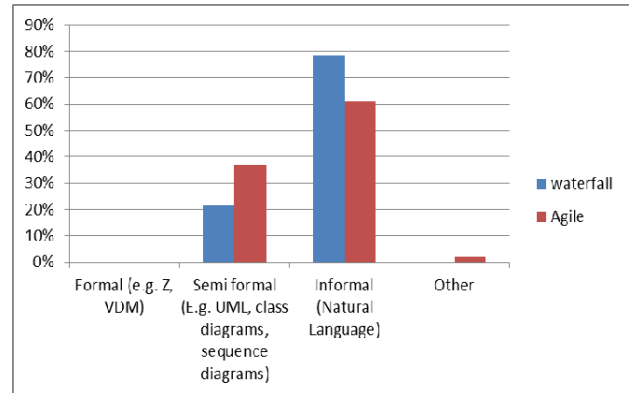


Figure 6. Requirements Formalisms used across agile vs. waterfall

C. Requirements Management

An interesting corollary to the formality discussion is the requirements review and inspection effort. We found that the majority of respondents are performing requirements inspections. (67% of agile projects comparing to 54% of waterfall). That group of respondents employed, on average, 2.29 techniques with team review as the most common technique for both agile and waterfall. In fact, both agile and waterfall were close in their preferences for inspection techniques. Ad hoc walk-throughs ranked second while Checklist ranked third for both approaches (see Figure 7). We also found that the variance with the level of satisfaction with requirements validation efforts between agile and waterfall was not big (56% for agile comparing to 52% for waterfall). Automated requirements inspection using tools remain unpopular for both agile and waterfall projects.

Requirements traceability is another sub-discipline of requirements management which is concerned with documenting the life of a requirement and providing bi-directional traceability between various associated requirements. We observed that 85% of the agile respondents reported a satisfaction with the efforts to maintain proper requirements traceability during their projects; comparing to 68% for the waterfall projects.

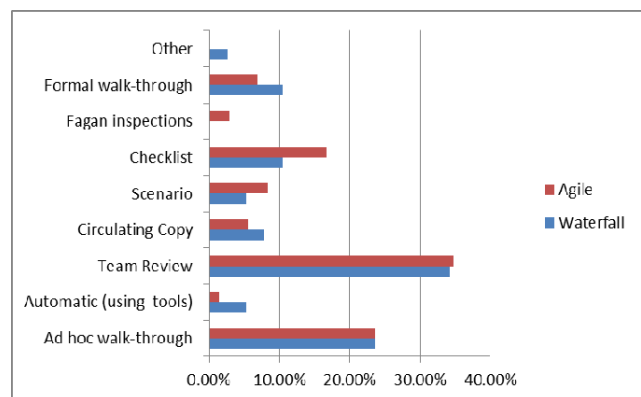


Figure 7. Requirements inspection techniques used across agile vs. waterfall

D. Requirements Effort Estimation

Out of the 60% of those who reported on performing estimation for the size of requirements or the effort of building them, 62% reported on taking into account the Non-Functional Requirements (NFRs) during the size / effort estimation.

When we broke these numbers down across the SDLC practices; we found that 67% of agile projects reported on performing estimation (comparing to 45% for waterfall); and then 68% of this population considered NFRs during the estimation (comparing to 40% for waterfall). This was a surprising finding; as there is a common belief that agile methodologies deal less with NFRs compared to waterfall [6]. One reason for this belief is that NFRs are not always apparent when dealing with requirements functionality only (through user stories).

In general, for those who conducted estimation for the size/ effort, “Expert Judgments” was the most popular technique (24%). Expert judgment was the dominant technique for waterfall projects. While it was still popular for agile projects (ranked second for these projects), we see an emerge of “Group estimation” techniques (e.g. planning poker, wideband Delphi) (see Figure 8).

We also see an emerge of “story points” in agile projects (ranked third) compared to their usage in waterfall projects (ranked 5th). Function points and COCOMO are not popular techniques in both waterfall and agile projects.

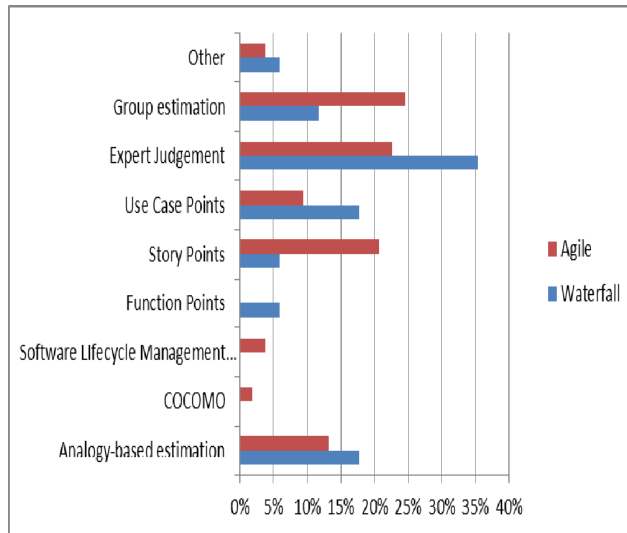


Figure 8. Effort estimation techniques used across agile vs. waterfall

We further linked the question of estimation techniques to whether the duration of the project was within schedule (see Figure 9). Considering the results from this linkage and since waterfall projects reported more reliance on experts judgment and analogy based approaches compared to the agile projects that reported more reliance on Group estimation and Story Points; we believe this may be an important factor explaining our other finding on agile outperforming (compared to waterfall) in project’s duration and budget estimation.

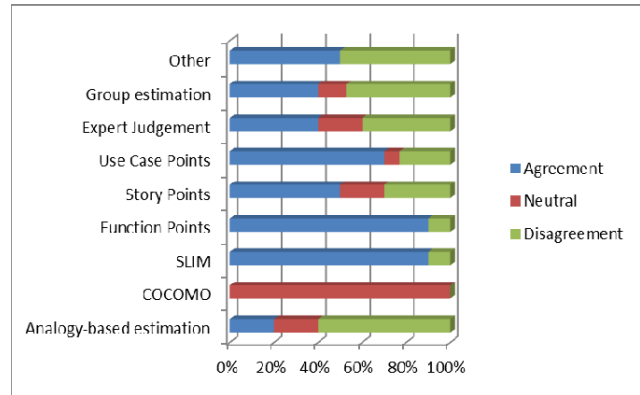


Figure 9. Link between the effort estimation technique and the statement: “The duration of the project was within schedule”

E. Tools

A concept closely related to SDLC is Application Lifecycle Management (ALM). Tools are capable of managing ALM activities from initial concept, through requirements analysis, development, and testing. Software Engineering literature, as well as industry advertisement, indicates that adoption of ALM tools improve the quality of the RE effort and enforce the usage of SDLC methodologies [13]. This same literature proposes that the application of these methodologies coincide with a greater rate of project success. In terms of RE, these tools are coded to apply certain RE techniques; the techniques applied vary by product.

Overall, 64% of respondents reported that an ALM application was used. We observed that the usage of ALM tools in agile projects was 140% greater than in the waterfall projects. This was an interesting finding considering that agile and ALM may appear incompatible [14]. While both involve the management of processes, their approaches differ in many ways. Agile emphasizes flexibility and quick response to change, whereas ALM is based on tightly controlled processes that do not adapt well to changing business requirements and needs.

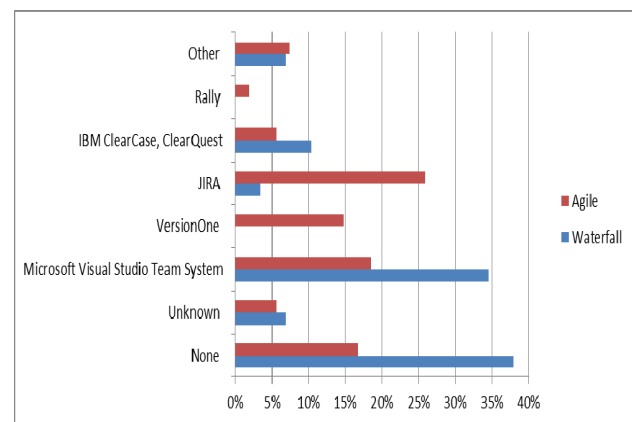


Figure 10. ALM tools usage across agile vs. waterfall

About third of the overall respondents in this survey who adopted an ALM, used Microsoft Visual Studio, and about 20% of them used JIRA. Other responses included “Bugzilla” and “Home Grown”. It makes a perfect sense to see that “VerizonOne”; which is a project management tool that was designed for eXtreme Programming and Scrum, is almost only used in the agile projects (See Figure 10).

F. Software Quality and Productivity

Respondents were asked a series of questions meant to assess the level of quality and productivity the project achieved.

Seventy four percent of respondents agreed that capabilities of the finished product fitted well with customer

or user needs. However the majority of respondents also indicated that the projects represented in this study took longer than the respondents had expected to deliver (52%) and with over budget (55%). Interestingly, 55% of respondents also indicated that the project couldn’t have been completed any faster, even if that would have meant a compromised product quality.

Overall, agile projects outperformed the waterfall projects in maintaining the project within the schedule and budget (see Figure 11). On the other hand, the variance between the impact of the two SDLCs on the final product’s capabilities and quality was not significant.

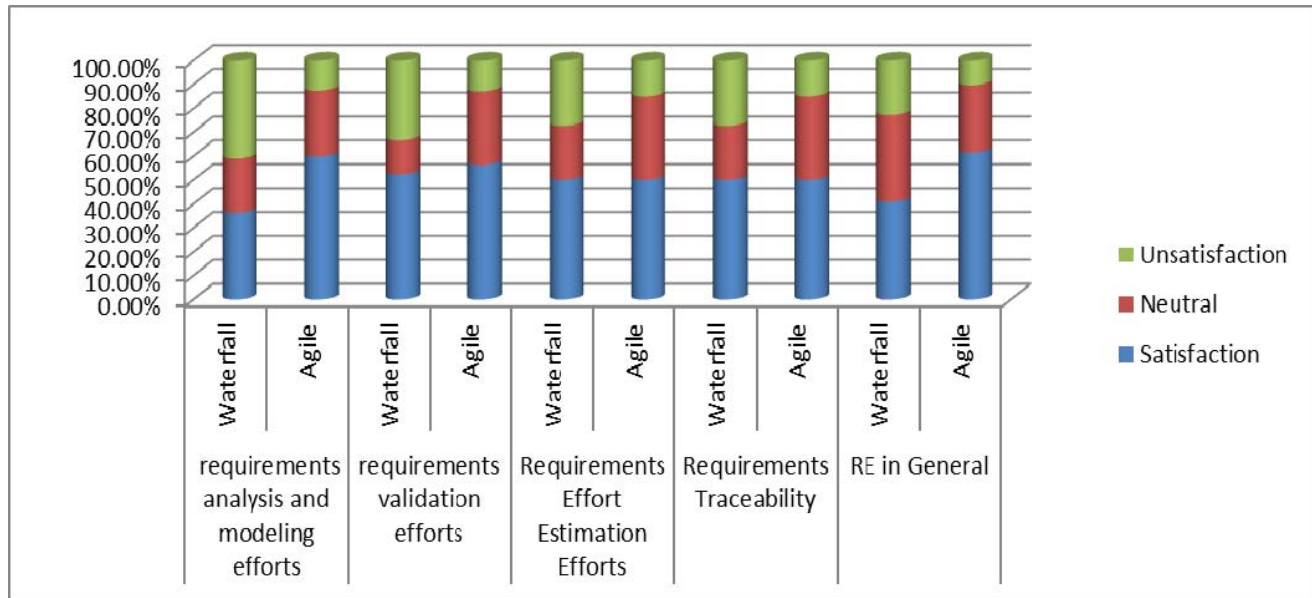


Figure 11. Reported level of satisfaction on the productivity and final product’s capabilities, qualities

IV. RELATED WORK

In [12], the authors compare traditional requirements engineering approaches and agile software development. They analyzed commonalities and differences of both approaches. Their findings were that the RE phases is not very clear in agile: they are merged and repeated in each iteration. Also, documentation is a main difference between the RE process and agile methods: There is a lack of documentation in agile. Changes being traceable are regarded important in RE, and agile methods also manage the requirements well with index cards. Traditional RE does not have a good way of stakeholder involvement as agile methods: customers are only involved in early stages in RE while they are involved during the whole process under agile. The authors then recommended that a minimum of documentation should be ensured in an agile environment as a possible way for the agile to benefit from the RE methods.

Scott Ambler [15] suggests a set of best practices for RE using agile methods. Many of the practices follow directly from the principles behind the agile Manifesto. Ambler also

suggests using such artifacts as CRCs, acceptance tests, business rule definitions, change cases, dataflow diagrams, user interfaces, use cases, prototypes, features and scenarios, use case diagrams, and user stories to model requirements. These elements can be added to the software requirements specification document along with the user stories. For requirements elicitation he suggests using interviews, focus groups, JAD, legacy code analysis, ethnographic observation domain analysis, and having the customer on site at all times.

Bose, Kurhekar and Ghoshal [1] suggested that agile requirements are effective to get continuous feedback from customers while the limits of agile methods are still not defined well. They suggested that the requirements management should be adopted to ensure the traceability, which is important when the requirements are likely to be changed in an agile method. The verification of early requirements descriptions being included are also proposed.

The literature is rich with other survey studies on RE practices, development needs, and preferred ways that target

local geographic location (e.g. Finland [16], Chile [17], and Malaysia [18]).

In [11] the authors (Neill and Laplante) presented results of a comprehensive survey that was conducted in 2002 showing requirements engineering practices across a broad range of industries and projects types. The results were surprising in that they indicated, among other things, that the waterfall model was still widely used and that various techniques associated with agile development were not widely employed. Unpublished results from a similar survey in 2008 [7] indicated that the findings from 2002 had remained largely unchanged. The 2002 survey results were highly cited (181 times via Google Scholar), and seemed to provide a template for more focused requirements surveys. For example Khurum et al [19] conducted a brief survey to uncover challenges in organizations to effective requirements engineering, Chernak surveyed a small group of companies to determine the prevalence of requirements reuse and Verner et al [20] sought to uncover specific issues with respect to requirements management. But since 2002 and again in 2008 the results of no other comprehensive surveys of requirements engineering practices were published. To remedy this deficiency, and provide useful data to other researchers, we updated and reprised the 2002 and 2008 surveys. The present survey discussed in this paper includes responses from a broader geographic base including international participants. In a related paper [21] we reported on comparing the results from this survey with the ones from 2002 and 2008.

V. CONCLUSION

Throughout this paper we have reported on the current landscape of practice in requirements elicitation, analysis and

presentation, management, effort estimation and tools with respect to agile methodologies in comparison with the traditional waterfall. The comparison has revealed some interesting trends including the continued rise of agile methods and their accompanying practices. More revealing, however, is that some techniques and paradigms have still not risen to dominance as we would expect. Object-oriented analysis and design and the UML, for example, are reportedly far less common than we anticipated, which could be considered disappointing given the improvements in end-product capability and ease of use that such techniques reportedly provide. Some other salient observations that emerge from this survey are:

- A number of RE practices show no significant difference between agile and waterfall (e.g. Requirements validation).

- A number of practices were surprising findings considering the background on agile (e.g. usage of ALM, OO analysis, considering NFRs in effort estimation).

- Even though some techniques were developed specifically for a particular SDLC methodology (e.g. user stories for agile); it is interesting to see these techniques finding their way to the other SDLC methodologies (e.g. user stories in waterfall).

- Overall respondents expressed greater satisfaction with regards to the efforts of RE practices applied in their agile projects compared to the waterfall projects (see Figure 12).

Our hope is that researchers will use this data to corroborate their own research and to motivate follow-up research studies.

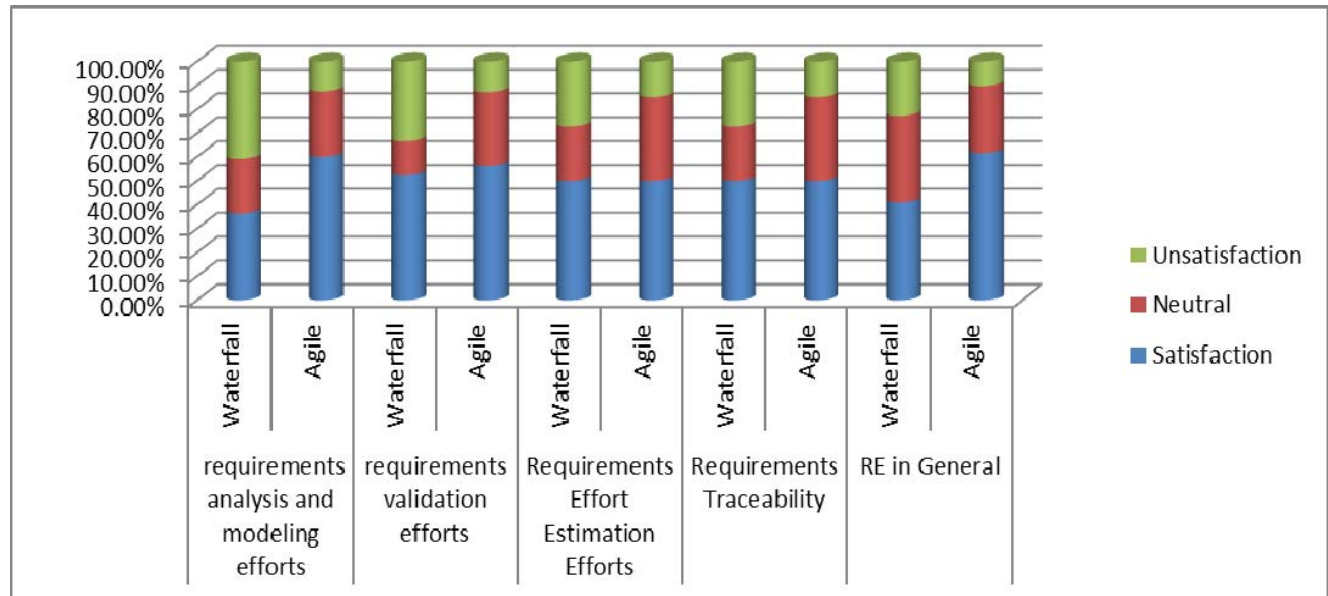


Figure 12. Reported level of satisfaction in RE practices across agile vs. waterfall

REFERENCES

- [1] S. Bose, M. Kurhekar, and G. Joydip, "agile methodology in Requirements Engineering", SETLabs Briefings Online, <http://www.infosys.com/infosys-labs/publications/Documents/agile-requirements-engineering.pdf>, last accessed: February 2014.
- [2] T. Huston, "what is software testing? A Brief Overview of the agile Approach to Software Development and Testing", <http://smartbear.com/products/qa-tools/what-is-agile-testing/>, last accessed: February 2014.
- [3] agile Manifesto, <http://agilemanifesto.org/>, last accessed: February 2014.
- [4] K. Orr, "agile Requirements: Opportunity or Oxymoron?", IEEE Software, Volume:21, Issue: 3, May 2004.
- [5] E. Letier, J. Araujo, R. Gacitua, and P. Sawyer, "First International Workshop on agile Requirements Engineering", In conjunction with the European Conference on Object Oriented Programming (ECOOP), Lancaster, UK, July 2011.
- [6] L. Williams, "agile Requirements Elicitation", <http://agile.csc.ncsu.edu/SEMaterals/agileRE.pdf>, last accesses: February 2014.
- [7] V. Marinelli, "An Analysis of Current Trends in Requirements Engineering Practice," Master of Software Engineering Professional Paper, Penn State University, Great Valley School of Graduate Professional Studies, Malvern, PA, 2008.
- [8] W. Cunningham, "The WyCash Portfolio Management System", technical report in Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'92), Seventh Annual Conference, Vancouver, British Columbia, Canada, October 18-22, 1992.
- [9] A. Sillitti and G. Succi, "Requirements Engineering for agile methods", Engineering and Managing Software Requirements, Springer Berlin Heidelberg, pp 309-326, 2005.
- [10] K. Beck and M. Fowler, "Planning Extreme Programming", Boston, MA: Addison Wesley, 2001.
- [11] N. Colin and P. Laplante, "Requirements Engineering: The State of the Practice", Software, vol. 20, no. 6, pp. 40-46, November 2003.
- [12] F. Paetsch, A. Eberlein, and F. Maurer, "Requirements Engineering and Agile Software Development", Proceedings of the IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, page 308--313. Linz, Austria, 2003.
- [13] D. West, "Integrated ALM Tools Are Fundamental to Success", <http://www.infoq.com/articles/Integrated-ALM>; last accesses: February 2014.
- [14] G. Owen, "agile and ALM: A marriage made in heaven?" <http://searchsoftwarequality.techtarget.com/tip/agile-and-ALM-A-marriage-made-in-heaven>, last accesses : February 2014.
- [15] S. Ambler, "Agile Requirements Chagement Management", <http://agilemodeling.com/essays/changeManagement.htm>, last accesses February 2014.
- [16] U. Nikula, J. Sajaniemi, and H. Kälviäinen, "A State-of-the-Practice Survey on Requirements Engineering in Small- and Medium-Sized Enterprises", Research report, Telecom Business Research Center Lappeenranta, 2000.
- [17] A. Quispe, M. Marques, L. Silvestre, S.F. Ochoa, R. Robbes, "Requirements Engineering Practices in Very Small Software Enterprises: A Diagnostic Study", CCC 2010, Proceedings of the XXIX International Conference of the Chilean Computer Science Society, Antofagasta, Chile, 15-19 November 2010.
- [18] B. Solemon, S. Sahibuddin, A. A. Abd Ghani, "Requirements Engineering Problems and Practices in Software Companies: An Industrial Survey", Advances in Software Engineering Communications in Computer and Information Science Volume 59, 2009, pp 70-77.
- [19] M. Khurum, N. Uppalapati, and R.C Veeramachaneni, "Software Requirements Triage and Selection: State-of-the-Art and State-of-Practice", Software Engineering Conference (APSEC), 2012 19th Asia-Pacific, vol.1, no., pp.416,421, 4-7 Dec. 2012.
- [20] J. Verner, K. Cox, S. Bleistein, and N. Cerpa, "Requirements engineering and software project success: an industrial survey in Australia and the US," Australasian Journal of Information Systems, 2007, 13(1).
- [21] M. Kassab, C. Neill, and P. Laplante, "State of practice in requirements engineering: contemporary data", Innovation in Systems and Software Engineering: A NASA journal, 2014, DOI: 10.1007/s11334-014-0232-.